Binding as Sets of Scopes

Matthew Flatt
University of Utah, USA
mflatt@cs.utah.edu

Abstract

Our new macro expander for Racket builds on a novel approach to hygiene. Instead of basing macro expansion on variable renamings that are mediated by expansion history, our new expander tracks binding through a *set of scopes* that an identifier acquires from both binding forms and macro expansions. The resulting model of macro expansion is simpler and more uniform than one based on renaming, and it is sufficiently compatible with Racket's old expander to be practical.

Categories and Subject Descriptors D.3.3 [*Programming Languages*]: Language Constructs and Features

Keywords Macros, hygiene, binding, scope

1. Introduction: Lexical vs. Macro Scope

Racket supports a family of languages—including the main Racket language, Typed Racket, teaching languages, a documentation language, an Algol 60 implementation, and more—where the languages interoperate in natural and useful ways. Furthermore, programmers can create new languages and language extensions with the same level of support and interoperability as the predefined languages. Racket's support for multiple languages and extensions is enabled by its macro system, which provides a representation of syntax fragments and constructs for manipulating and composing those fragments.

Racket's representation of syntax builds on a long line of work on macros in Lisp and Scheme (Kohlbecker et al. 1986; Kohlbecker and Wand 1987; Clinger and Rees 1991; Dybvig et al. 1993). A result of that line of work is often referenced by the shorthand of *hygienic macro expansion*, meaning that macros can both introduce bindings and refer to binding from the macro's definition context, and macro expansion will not trigger accidental name capture in the way that naive textual expansion would. Although Racket does not restrict language extension to forms that can be expressed as hygienic macros, a representation of syntax fragments that supports hygienic expansion is an essential ingredient to more general, composable language extensions.

Roughly, hygienic macro expansion is desirable for the same reason as lexical scope: both enable local reasoning about binding so that program fragments compose reliably. The analogy suggests specifying hygienic macro expansion as a kind of translation into lexical-scope machinery. In that view, variables must be *renamed* to match the mechanisms of lexical scope as macro expansion proceeds. A specification of hygiene in terms of renaming accommodates simple binding forms well, but it becomes unwieldy for recursive definition contexts (Flatt et al. 2012, section 3.8), especially for contexts that allow a mixture of macro and non-macro definitions. The renaming approach is also difficult to implement compactly and efficiently in a macro system that supports "hygiene bending" operations, such as `datum->syntax`, because a history of renamings must be recorded for replay on an arbitrary symbol.

In a new macro expander for Racket, we discard the renaming approach and start with a generalized idea of macro scope, where lexical scope is just a special case of macro scope when applied to a language without macros. Roughly, every binding form and macro expansion creates a *scope*, and each fragment of syntax acquires a *set of scopes* that determines binding of identifiers within the fragment. In a language without macros, each scope set is identifiable by a single innermost scope. In a language with macros, identifiers acquire scope sets that overlap in more general ways.

Our experience is that this set-of-scopes model is simpler to use than the old macro expander, especially for macros that work with recursive-definition contexts or create unusual binding patterns. Along similar lines, the expander's implementation is simpler than the old one based on renaming, and the implementation avoids bugs that were difficult to repair in the old expander. Finally, the new macro expander is able to provide more helpful debugging information when binding resolution fails. All of these benefits reflect the way that scope sets are precise enough for specification but abstract enough to allow high-level reasoning.

This change to the expander's underlying model of binding can affect the meaning of existing Racket macros. A small amount of incompatibility seems acceptable and even desirable if it enables easier reasoning overall. Drastic incompatibilities would be suspect, however, both because the old expander has proven effective and because large changes to code base would be impractical. Consistent with those aims, purely pattern-based macros work with the new expander the same as with the old one, except for unusual macro patterns within a recursive definition context. More generally, our experiments indicate that the majority of existing Racket macros work unmodified, and other macros can be adapted with reasonable effort.

2. Background: Scope and Macros

An essential consequence of hygienic macro expansion is to enable macro definitions via *patterns* and *templates*—also known as *macros by example* (Kohlbecker and Wand 1987; Clinger and Rees 1991). Although pattern-based macros are limited in various ways, a treatment of binding that can accommodate patterns and templates is key to the overall expressiveness of a hygienic macro system, even for macros that are implemented with more general constructs.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from Permissions@acm.org.

Copyright is held by the owner/author(s). Publication rights licensed to ACM.

POPL'16, January 20–22, 2016, St. Petersburg, FL, USA
ACM. 978-1-4503-3549-2/16/01...$15.00
http://dx.doi.org/10.1145/2837614.2837620

As an example of a pattern-based macro, suppose that a Racket library implements a Java-like object system and provides a `send` form, where

```
(send a-pt rotate 90)
```

evaluates `a-pt` to an object, locates a function mapped to the symbol `'rotate` within the object, and calls the function as a method by providing the object itself followed by the argument `90`. Assuming a `lookup-method` function that locates a method within an object, the `send` form can be implemented by a pattern-based macro as follows:

```
(define-syntax-rule (send obj-expr method-name arg)
  (let ([obj obj-expr])
    ((lookup-method obj 'method-name) obj arg)))
```

The `define-syntax-rule` form declares a *pattern* that is keyed on an initial identifier; in this case, the pattern is keyed on `send`. The remaining identifiers in the parenthesized `send` pattern are *pattern variables*. The second part of the definition specifies a *template* that replaces a match of the pattern, where each use of a pattern variable in the template is replaced with the corresponding part of the match.

With this definition, the example use of `send` above matches the pattern with `a-pt` as *`obj-expr`*, `rotate` as *`method-name`*, and `90` as *`arg`*, so the `send` use expands to

```
(let ([obj a-pt])
  ((lookup-method obj 'rotate) obj 90))
```

Hygienic macro expansion ensures that the identifier `obj` is not accidentally referenced in an expression that replaces *`arg`* in a use of `send` (Kohlbecker et al. 1986). For example, the body of

```
(lambda (obj)
  (send a-pt same? obj))
```

must call the `same?` method of `a-pt` with the function argument `obj`, and not with `a-pt` itself as bound to `obj` in the macro template for `send`. Along similar lines, a local binding of `lookup-method` at a use site of `send` must not affect the meaning of `lookup-method` in `send`'s template. That is,

```
(let ([lookup-method #f])
  (send a-pt rotate 90))
```

should still call the `rotate` method of `a-pt`.

Macros can be bound locally, and macros can even expand to definitions of macros. For example, suppose that the library also provides a `with-method` form that performs a method lookup just once for multiple sends:

```
(with-method ([rot-a-pt (a-pt rotate)]) ; find rotate
  (for ([i 1000000])
    (rot-a-pt 90))) ; send rotate to point many times
```

The implementation of `with-method` can make `rot-a-pt` a local macro binding, where a use of `rot-a-pt` expands to a function call with `a-pt` added as the first argument to the function. That is, the full expansion is

```
(let ([obj a-pt])
  (let ([rot-a-pt-m (lookup-method obj 'rotate)])
    (for ([i 1000000])
      (rot-a-pt-m obj 90))))
```

but the intermediate expansion is

```
(let ([obj a-pt])
  (let ([rot-a-pt-m (lookup-method obj 'rotate)])
    (let-syntax ([rot-a-pt (syntax-rules ()
                             [(rot-a-pt arg)
                              (rot-a-pt-m obj arg)])])
      (for ([i 1000000])
        (rot-a-pt 90)))))
```

where `let-syntax` locally binds the macro `rot-a-pt`. The macro is implemented by a `syntax-rules` form that produces an anonymous pattern-based macro (in the same way that `lambda` produces an anonymous function).

In other words, `with-method` is a binding form, it is a macro-generating macro, it relies on local-macro binding, and the macro that it generates refers to a private binding `obj` that is also macro-introduced. Nevertheless, `with-method` is straightforwardly implemented as a pattern-based macro:

```
(define-syntax-rule
  (with-method ([local-id (obj-expr method-name)])
    body)
  (let ([obj obj-expr])
    (let ([method (lookup-method obj 'method-name)])
      (let-syntax ([local-id (syntax-rules ()
                               [(local-id arg)
                                (method obj arg)])])
        body))))
```

Note that the `obj` binding cannot be given a permanently distinct name within `with-method`. A distinct name must be generated for each use of `with-method`, so that nested uses create local macros that reference the correct `obj`.

In general, the necessary bindings or even the binding structure of a macro's expansion cannot be predicted in advance of expanding the macro. For example, the `let` identifier that starts the `with-method` template could be replaced with a macro argument, so that either `let` or, say, a lazy variant of `let` could be supplied to the macro. The expander must accommodate such macros by delaying binding decisions as long as possible. Meanwhile, the expander must accumulate information about the origin of identifiers to enable correct binding decisions.

Even with additional complexities—where the macro-generated macro is itself a binding form, where uses can be nested so the different uses of the generated macro must have distinct bindings, and so on—pattern-based macros support implementations that are essentially specifications (Kohlbecker and Wand 1987). A naive approach to macros and binding fails to accommodate the specifications (Adams 2015, sections 4.2-4.5), while existing formalizations of suitable binding rules detour into concepts of marks and renamings that are distant from the programmer's sense of the specification.

The details of a formalization matter more when moving beyond pattern-matching macros to *procedural macros*, where the expansion of a macro can be implemented by an arbitrary compile-time function. The `syntax-case` and `syntax` forms provide the pattern-matching and template-construction facilities, respectively, of `syntax-rules`, but they work as expressions within a compile-time function (Dybvig et al. 1993). This combination allows a smooth transition from pattern-based macros to procedural macros for cases where more flexibility is needed. In fact, `syntax-rules` is itself simply a macro that expands to a procedure:

```
(define-syntax-rule (syntax-rules literals
                      [pattern template] ...)
  (lambda (stx)
    (syntax-case stx literals
      [pattern #'template] ; #'_ is short for (syntax _)
      ...)))
```

Besides allowing arbitrary computation mixed with pattern matching and template construction, the `syntax-case` system provides operations for manipulating program representations as *syntax objects*. Those operations include "bending" hygiene by attaching the binding context of one syntax object to another. For example, a macro might accept an identifier `point` and synthesize the identifier `make-point`, giving the new identifier the same con-

text as `point` so that `make-point` behaves as if it appeared in the same source location with respect to binding.

Racket provides an especially rich set of operations on syntax objects to enable macros that compose and cooperate (Flatt et al. 2012). Racket's macro system also relies on a module layer that prevents interference between run-time and compile-time phases of a program, since interference would make macros compose less reliably (Flatt 2002). Finally, modules can be nested and macro-generated, which enables macros and modules to implement facets of a program that have different instantiation times—such as the program's run-time code, its tests, and its configuration metadata (Flatt 2013). The module-level facets of Racket's macro system are, at best, awkwardly accommodated by existing models of macro binding; those models are designed for expression-level binding, where α-renaming is straightforward, while modules address a more global space of mutually recursive macro and variable definitions. A goal of our new binding model is to more simply and directly account for such definition contexts.

3. Scope Sets for Pattern-Based Macros

Like previous models of macro expansion, our set-of-scopes expander operates on a program from the outside in. The expander detects bindings, macro uses, and references as part of the outside-to-inside traversal. The difference in our expander is the way that bindings and macro expansions are recorded and attached to syntax fragments during expansion.

3.1 Scope Sets

A *scope* corresponds to a binding context, and every identifier in a program has a set of scopes. For example, if we treat `let` and `lambda` as primitive binding forms, then in the fully expanded expression

```
(let ([x 1])
  (lambda (y)
    z))
```

the `let` form corresponds to a scope a_{let}, and the `lambda` form corresponds to b_{lam}. That is, everything in the `let`'s body is in a_{let}, and everything in the inner `lambda`'s body is in b_{lam}; the set of scopes associated with `z` is $\{a_{let}, b_{lam}\}$. (Notation: the subscripts on a_{let} and b_{lam} are just part of the names that we use to refer to abstract scope tokens; they have no meaning beyond indicating the scope's origin.)

In a macro-extensible language, expanding a use of a macro creates a new scope in the same way that a binding form creates a new scope. Starting with

```
(let ([x 1])
  (let-syntax ([m (syntax-rules ()
                    [(m) x])])
    (lambda (x)
      (m))))
```

the right-hand side of the `m` binding has the scope set $\{a_{let}\}$, while the final `m` has scope set $\{a_{let}, b_{ls}, c_{lam}\}$ corresponding to the `let`, `let-syntax`, and `lambda` forms. We can write the scope sets next to each `x` and `m` at the point where macro expansion reaches the `(m)` form:

```
(let ([x^{a_let} 1])
  (let-syntax ([m^{a_let, b_ls} (syntax-rules ()
                                  [(m) #'x^{a_let}])])
    (lambda (x^{a_let, b_ls, c_lam})
      (m^{a_let, b_ls, c_lam}))))
```

The expansion of `(m)` produces `x` with the scope set $\{a_{let}, d_{intro}\}$, where d_{intro} is a new scope for identifiers that are introduced by the macro's expansion:

```
(let ([x^{a_let} 1])
  (let-syntax ([m^{a_let, b_ls} (syntax-rules ()
                                  [(m) #'x^{a_let}])])
    (lambda (x^{a_let, b_ls, c_lam})
      x^{a_let, d_intro})))
```

The absence of c_{lam} on the final `x` explains why it doesn't refer to the inner binding of `x`. At the same time, if a different `m` places a macro-introduced `x` in a binding position around an `x` from a macro use `(m x)`, the `x` from the use is not macro-introduced and doesn't have the scope d_{intro}, so it wouldn't refer to the macro-introduced binding.

Lexical scoping corresponds to sets that are constrained to a particular shape: For any given set, there's a single scope s that implies all the others (i.e., the ones around s in the program). As a result, s by itself is enough information to identify a binding for a given reference. We normally describe lexical scope in terms of the closest such s for some notion of "closest." Given scope sets instead of individual scopes, we can define "closest" as the largest relevant set.

More generally, we can define binding based on *subsets*: A reference's binding is found as one whose set of scopes is a subset of the reference's own scopes (in addition to having the same symbolic name). The advantage of using sets of scopes is that macro expansion creates scope sets that overlap in more general ways; there's not always a s that implies all the others. Absent a determining s, we can't identify a binding by a single scope, but we can identify it by a set of scopes.

If arbitrary sets of scopes are possible, then two different bindings might have overlapping scopes, neither might be a subset of the other, and both might be subsets of a particular reference's scope set. In that case, the reference is ambiguous. Creating an ambiguous reference with only pattern-based macros is possible, but it requires a definition context that supports mingled macro definitions and uses; we provide an example in section 3.5.

3.2 Bindings

When macro expansion encounters a primitive binding form, it

- creates a new scope;
- adds the scope to every identifier in binding position, as well as to the region where the bindings apply; and
- extends a global table that maps a $\langle$symbol, scope set$\rangle$ pair to a representation of a binding.

In a simplified language where bindings are local, an identifier with its scope set could be its own representation of a binding. In a more complete language, bindings can also refer to module imports. We therefore represent a local binding with a unique, opaque value (e.g., a gensym).

For example,

```
(let ([x 1])
  (let-syntax ([m (syntax-rules ()
                    [(m) x])])
    (lambda (x)
      (m))))
```

more precisely expands after several steps to

```
(let ([x^{a_let} 1])
  (let-syntax ([m^{a_let, b_ls}
                 (syntax-rules ()
                   [(m) #'x^{a_let}])])
    (lambda (x^{a_let, b_ls, c_lam})
      x^{a_let, d_intro})))
```

$x^{\{a_{let}\}}$	$\rightarrow$ x4
$m^{\{a_{let}, b_{ls}\}}$	$\rightarrow$ m8
$x^{\{a_{let}, b_{ls}, c_{lam}\}}$	$\rightarrow$ x16

where the compile-time environment along the way (not shown) maps `x4` to a variable, `m8` to a macro, and `x16` to another variable.

The reference $\texttt{x}^{\{a_{let}, d_{intro}\}}$ has the binding `x4`, because $\texttt{x}^{\{a_{let}\}}$ is the mapping for `x` in the binding table that has the largest subset of $\{a_{let}, d_{intro}\}$.

The distinction between the binding table and the compile-time environment is important for a purely "syntactic" view of binding, where a term can be expanded, manipulated, transferred to a new context, and then expanded again. Some approaches to macros, such as syntactic closures (Bawden and Rees 1988) and explicit renaming (Clinger 1991), tangle the binding and environment facets of expansion so that terms cannot be manipulated with the same flexibility.

The binding table can grow forever, but when a particular scope becomes unreachable (i.e., when no reachable syntax object includes a reference to the scope), then any mapping that includes the scope becomes unreachable. This weak mapping can be approximated by attaching the mapping to the scope, instead of using an actual global table. Any scope in a scope set can house the binding, since the binding can only be referenced using all of the scopes in the set. Attaching to the most recently allocated scope is a good heuristic, because the most recent scope is likely to be maximally distinguishing and have the shortest lifetime.

3.3 Recursive Macros and Use-Site Scopes

So far, our characterization of macro-invocation scopes works only for non-recursive macro definitions. To handle recursive macro definitions, in addition to a fresh scope to distinguish forms that are introduced by a macro, a fresh scope is needed to distinguish forms that are present at the macro use site.

Consider the following `letrec-syntax` expression, whose meaning depends on whether a use-site identifier captures a macro-introduced identifier:

```
(letrec-syntax ([identity (syntax-rules ()
                            [(_ misc-id)
                             (lambda (x)
                               (let ([misc-id 'other])
                                 x))])])
  (identity x))
```

Assuming that the `letrec-syntax` form creates a scope a_{ls}, the scope must be added to both the right-hand side and body of the `letrec-syntax` form to create a recursive binding:

```
(letrec-syntax ([identity (syntax-rules ()
                            [(_ misc-id)
                             (lambda (x^{a_ls})
                               (let ([misc-id 'other])
                                 x^{a_ls}))])])
  (identity x^{a_ls}))
```

If we create a scope only for introduced forms in a macro expansion, then expanding `(identity` $\texttt{x}^{\{a_{ls}\}}$`)` creates the scope set b_{intro} and produces

```
(lambda (x^{a_ls, b_intro})
  (let ([x^{a_ls} 'other])
    x^{a_ls, b_intro}))
```

where b_{intro} is added to each of the two introduced `x`s. The `lambda` introduces a new scope c_{lam}, and `let` introduces d_{let}, producing

```
(lambda (x^{a_ls, b_intro, c_lam})
  (let ([x^{a_ls, c_lam, d_let} 'other])
    x^{a_ls, b_intro, c_lam, d_let}))
```

At this point, the binding of the innermost `x` is ambiguous: $\{a_{ls}, b_{intro}, c_{lam}, d_{let}\}$ is a superset of both $\{a_{ls}, b_{intro}, c_{lam}\}$ and $\{a_{ls}, c_{lam}, d_{let}\}$, neither of which is a subset of the other. Instead, we want `x` to refer to the `lambda` binding.

Adding a scope for the macro-use site corrects this problem. If we call the use-site scope e_{use}, then we start with

```
(identity x^{a_ls, e_use})
```

which expands to

```
(lambda (x^{a_ls, b_intro})
  (let ([x^{a_ls, e_use} 'other])
    x^{a_ls, b_intro}))
```

which ends up as

```
(lambda (x^{a_ls, b_intro, c_lam})
  (let ([x^{a_ls, c_lam, d_let, e_use} 'other])
    x^{a_ls, b_intro, c_lam, d_let}))
```

There's no ambiguity, and the final `x` refers to the `lambda` binding as intended. In short, each macro expansion needs a use-site scope as the symmetric counterpart to the macro-introduction scope.

3.4 Use-Site Scopes and Macro-Generated Definitions

In a binding form such as `let` or `letrec`, bindings are clearly distinguished from uses by their positions within the syntactic form. In addition to these forms, Racket (like Scheme) supports definition contexts that mingle binding forms and expressions. For example, the body of a module contains a mixture of definitions and expressions, all in a single recursive scope. Definitions can include macro definitions, expressions can include uses of those same macros, and macro uses can even expand to further definitions.

With set-of-scopes macro expansion, macro definitions and uses within a single context interact badly with use-site scopes. For example, consider a `define-identity` macro that is intended to expand to a definition of a given identifier as the identity function:

```
(define-syntax-rule (define-identity id)
  (define id (lambda (x) x)))

(define-identity f)
(f 5)
```

If the expansion of `(define-identity f)` adds a scope to the use-site `f`, the resulting definition does not bind the `f` in `(f 5)`.

The underlying issue is that a definition context must treat use-site and introduced identifiers asymmetrically as binding identifiers. In

```
(define-syntax-rule (define-five misc-id)
  (begin
   (define misc-id 5)
   x))

(define-five x)
```

the introduced `x` should refer to an `x` that is defined in the enclosing scope, which turns out to be the same `x` that appears at the use site of `define-five`. But in

```
(define-syntax-rule (define-other-five misc-id)
  (begin
   (define x 5)
   misc-id))

(define-other-five x)
```

the `x` from the use site should not refer to the macro-introduced binding `x`.

To support macros that expand to definitions of given identifiers, a definition context must keep track of scopes created for macro uses, and it must remove those scopes from identifiers that end up in binding positions. In the `define-identity` and `define-five` examples, the use-site scope is removed from the binding identifiers `x` and `f`, so they are treated the same as if their definitions appeared directly in the source.

This special treatment of use-site scopes adds complexity to the macro expander, but it is of the kind of complexity that mutually recursive binding contexts create routinely (e.g., along the same lines as ensuring that variables are defined before they are referenced). Definition contexts in Racket have proven convenient and expressive enough to be worth the extra measure of complexity.

3.5 Ambiguous References

The combination of use-site scopes to solve local-binding problems (as in section 3.3) versus reverting use-site scopes to accommodate macro-generated definitions (as in section 3.4) creates the possibility of generating an identifier whose binding is ambiguous.

The following example defines `m` through a `def-m` macro, and it uses `m` in the same definition context:

```
(define-syntax-rule (def-m m given-x)
  (begin
    (define x 1)
    (define-syntax-rule (m)
      (begin
        (define given-x 2)
        x))))

(def-m m x)
(m)
```

The expansion, after splicing `begin`s, ends with an ambiguous reference:

```
(define-syntax-rule (def-m^{a_def} ....) ....)
(define x^{a_def, b_intro1} 1)
(define-syntax-rule (m^{a_def})
  (begin
    (define x^{a_def, b_use1} 2)
    x^{a_def, b_intro1}))
(define x^{a_def, c_intro2} 2)
x^{a_def, b_intro1, c_intro2}
```

The scope a_{def} corresponds to the definition context, b_{intro1} and b_{use1} correspond to the expansion of `def-m`, c_{intro2} corresponds to the expansion of `m`. The final reference to `x` is ambiguous, because it was introduced through both macro layers.

Unlike the ambiguity that is resolved by use-site scopes, this ambiguity arguably reflects an inherent ambiguity in the macro. Absent the `(define x 1)` definition generated by `def-m`, the final `x` reference should refer to the definition generated from `(define given-x 2)`; similarly, absent the definition generated from `(define given-x 2)`, the final `x` should refer to the one generated from `(define x 1)`. Neither of those definitions is more specific than the other, since they are generated by different macro invocations, so our new expander rejects the reference as ambiguous.

Our previous model of macro expansion to cover definition contexts (Flatt et al. 2012) would treat the final `x` always as a reference to the definition generated from `(define x 1)` and never to the definition generated from `(define given-x 2)`. So far, we have not encountered a practical example that exposes the difference between the expanders' treatment of pattern-based macros in definition contexts.

4. Procedural Macros and Modules

Although our set-of-scopes expander resolves bindings differently than in previous models, it still works by attaching information to identifiers, and so it can provide a smooth path from pattern-matching macros to procedural macros in the same way as `syntax-case` (Dybvig et al. 1993). Specifically, `(syntax` *`form`*`)` quotes the S-expression *`form`* while preserving its scope-set information, so that *`form`* can be used to construct the result of a macro.

More precisely, a primitive `(quote-syntax` *`form`*`)` quotes *`form`* with its scope sets in Racket. The derived `(syntax` *`form`*`)` detects uses of pattern variables and replaces them with their matches while quoting any non-pattern content in *`form`* with `quote-syntax`. A `(syntax` *`form`*`)` can be abbreviated `#'`*`form`*, and when *`form`* includes no pattern variables, `#'`*`form`* is equivalent to `(quote-syntax` *`form`*`)`. The quaisquoting variant `#` ` *`form`* (which uses a backquote instead of a regular quote) allows escapes within *`form`* as `#,`*`expr`*, which inserts the result of evaluating *`expr`* in place of the escape.

The result of a `quote-syntax` or `syntax` form is a *syntax object*. When a syntax object's S-expression component is just a symbol, then the syntax object is an *identifier*.

4.1 Identifier Comparisons with Scope Sets

Various compile-time functions work on syntax objects and identifiers. Two of the most commonly used functions are `free-identifier=?` and `bound-identifier=?`, each of which takes two identifiers. The `free-identifier=?` function is used to recognize a reference to a known binding, such as recognizing a use of `else` in a conditional. The `bound-identifier=?` function is used to check whether two identifiers would conflict as bindings in the same context, such as when a macro that expands to a binding form checks that identifiers in the macro use are suitably distinct.

These two functions are straightforward to implement with scope sets. A `free-identifier=?` comparison on identifiers checks whether the two identifiers have the same binding by consulting the global binding table. A `bound-identifier=?` comparison checks that two identifiers have exactly the same scope sets, independent of the binding table.

4.2 Local Bindings and Syntax Quoting

The set-of-scopes approach to binding works the same as previous models for macros that are purely pattern-based, but the set-of-scopes approach makes finer distinctions among identifiers than would be expected by existing procedural Racket macros that use `#'` or `quote-syntax`. To be consistent with the way that Racket macros have been written, `quote-syntax` must discard some scopes.

For example, in the macro

```
(lambda (stx)
  (let ([id #'x])
    #`(let ([x 1])
        #,id)))
```

the `x` that takes the place of `#,id` should refer to the binding `x` in the generated `let` form. The `x` identifier that is bound to `id`, however, is not in the scope that is created for the compile-time `let`:

```
(lambda (stx^{a_lam})
  (let ([id^{a_lam, b_let} #'x^{a_lam}])
    #`(let ([x^{a_lam, b_let} 1])
        #,id^{a_lam, b_let})))
```

If `quote-syntax` (implicit in `#` ` ) preserves all scopes on an identifier, then with set-of-scopes binding, the `x` that replaces `#,id` will not refer to the `x` in the generated `let`'s binding position.

It's tempting to think that the compile-time `let` should introduce a phase-specific scope that applies only for compile-time references, in which case it won't affect `x` as a run-time reference. That adjustment doesn't solve the problem in general, since a macro can generate compile-time bindings and references just as well as run-time bindings and references.

A solution is for the expansion of `quote-syntax` to discard certain scopes on its content. The discarded scopes are those from

binding forms that enclosed the `quote-syntax` form up to a phase crossing or module top-level, as well as any use-site scopes recorded for macro invocations within those binding forms. In the case of a `quote-syntax` form within a macro binding's right-hand side, those scopes cover all of the scopes introduced on the right-hand side of the macro binding.

The resulting macro system is different than the old Racket macro system. Experiments suggest that the vast majority of macro implementations work either way, but it's easy to construct an example that behaves differently:

```
(free-identifier=? (let ([x 1]) #'x)
                   #'x)
```

In Racket's old macro system, the result is `#f`. The set-of-scopes system with a scope-pruning `quote-syntax` produces `#t`, instead, because the `let`-generated scope is stripped away from `#'x`.

If `quote-syntax` did not prune scopes, then not only would the result above be `#f`, but `bound-identifier=?` would produce `#f` for both `(let ([x 1]) #'x)` and `(let ([y 1]) #'x)`. Those results reflect the switch to attaching identifier-independent scopes to identifiers, instead of attaching identifier-specific renamings.

Arguably, the issue here is the way that pieces of syntax from different local scopes are placed into the same result syntax object, with the expectation that all the pieces are treated the same way. In other words, Racket programmers have gotten used to an unusual variant of `quote-syntax`, and most macros could be written just as well with a non-pruning variant.

Supplying a second, non-pruning variant of `quote-syntax` poses no problems. Our set-of-scopes implementation for Racket implements the non-pruning variant when a `#:local` keyword is added to a `quote-syntax` form. For example,

```
(free-identifier=? (let ([x 1])
                     (quote-syntax x #:local))
                   (quote-syntax x #:local))
```

produces `#f` instead of `#t`, because the scope introduced by `let` is preserved in the body's syntax object. The non-pruning variant of `quote-syntax` is useful for embedding references in a program's full expansion that are meant to be inspected by tools other than the Racket compiler; Typed Racket's implementation uses the `#:local` variant of `quote-syntax` to embed type declarations (including declarations for local bindings) in a program's expansion for use by its type checker.

4.3 Ensuring Distinct Bindings

A Racket macro's implementation can arrange for an identifier introduced by a macro expansion to have an empty scope set.[1] More generally, a macro can arrange for identifiers that are introduced in different contexts to have the same symbol and scope set. If those identifiers appear as bindings via `lambda`, `let`, or `let-syntax`, then the new scope created for the binding form will ensure that the different identifiers produce different bindings. That is, the binding scope is always created after any expansion that introduced the bound identifier, so all bindings are kept distinct by those different binding scopes.

For example, assuming that `make-scopeless` creates an identifier that has no scopes in an expansion, then the `let-x` forms in

```
(define-syntax (let-x stx)
  (syntax-case stx ()
    [(_ rhs body)
     #`(let ([#,(make-scopeless 'x) rhs])
         body)]))

(let-x 5
  (let-x 6
    0))
```

create intermediate `x` identifiers that each have an empty scope set, but the full expansion becomes

```
(let ([x^{a_let} 5])
  (let ([x^{b_let} 6])
    0))
```

where a_{let} and b_{let} are created by each `let` (as a primitive binding form), and they distinguish the different `x` bindings.

In a definition context (see section 3.4), macro expansion can introduce an identifier to a binding position *after* the scope for the definition context is created (and after that scope is applied to the definition context's original content). That ordering risks a collision among bindings in different definition contexts, where identifiers introduced into different definition contexts all have the same symbol and set of scopes.

For example, using a `block` form that creates a definition context and that we treat here as a primitive form, the uses of `def-x` in

```
(define-syntax (def-x stx)
  (syntax-case stx ()
    [(_ rhs)
     #`(define #,(make-scopeless 'x) rhs)]))

(block
  (define y 1)
  (def-x 5))
(block
  (define y 2)
  (def-x 6))
```

risk expanding as

```
(block
  (define y^{a_def} 1)
  (define x^{} 5))
(block
  (define y^{b_def} 2)
  (define x^{} 6))
```

with conflicting bindings of `x` for the empty scope set.

To avoid the possibility of such collisions, in a definition context that supports both definitions and macro expansion, the context is represented by a pair of scopes: an *outside-edge scope* that is added to the original content of the definition context, and an *inside-edge scope* that is added to everything that appears in the definition context through macro expansion. The outside-edge scope distinguishes original identifiers from macro-introduced identifiers, while the inside-edge scope ensures that every binding created for the definition context is distinct from all other bindings.

Thus, the preceding example expands as

```
(block
  (define y^{a_out, a_in} 1)
  (define x^{a_in} 5))
(block
  (define y^{b_out, b_in} 2)
  (define x^{b_in} 6))
```

where the inside-edge scopes a_{in} and b_{in} distinguish the two `x` bindings. Meanwhile, if the definitions of `y` instead used the name `x`, they would remain distinguished from the macro-introduced `x`s by the outside-edge scopes a_{out} and b_{out}.

[1] Avoiding a macro-introduction scope involves using a `syntax-local-introduce` function.

4.4 First-Class Definition Contexts

Racket exposes the expander's support for definition contexts (see section 3.4) so that new macros can support definition contexts while potentially changing the meaning of a macro or variable definition. For example, the `class` macro allows local macro definitions in the `class` body while it rewrites specified function definitions to methods and other variable definitions to fields. The `unit` form similarly rewrites variable definitions to a mixture of private and exported definitions with a component.

Implementing a definition context starts with a call to `syntax-local-make-definition-context`, which creates a first-class (at compile time) value that represents the definition context. A macro can force expansion of forms in the definition context, it can add variable bindings to the definition context, and it can add compile-time bindings and values that are referenced by further macro expansion within the definition context. To a first approximation, a first-class definition context corresponds to an inside-edge scope that is added to any form expanded within the definition context and that houses the definition context's bindings. A definition context also has a compile-time environment frame (extending the context of the macro use) to house the mapping of bindings to variables and compile-time values.

Like other definition contexts (see section 3.4), the compile-time environment must track use-site scopes that are generated for macro expansions within a first-class definition context. If the macro moves any identifier into a binding position in the overall expansion, then the macro normally must remove accumulated use-site scopes (for the current definition context only) by applying `syntax-local-identifier-as-binding` to the identifier. For example, the `unit` form implements a definition context that is similar to the body of a `lambda`, but variables are internally transformed to support mutually recursive references across unit boundaries.

```
(unit (import)
      (export)
 (define x 1)
 x)
```

In this example, `(define x 1)` is expanded to `(define-values (x) 1)` with a use-site scope on `x`, but the intent is for this definition of `x` to capture the reference at the end of the `unit` form. If the `unit` macro simply moved the binding `x` into a `letrec` right-hand side, the `x` would not capture the final `x` as moved into the `letrec` body; the use-site scope on the definition's `x` would prevent it from capturing the use. The solution is for the `unit` macro to apply `syntax-local-identifier-as-binding` to the definition's `x` before using it as a `letrec` binding. Macros that use a definition context and `bound-identifier=?` must similarly apply `syntax-local-identifier-as-binding` to identifiers before comparing them with `bound-identifier=?`.

Even if a macro does not create a first-class definition context, some care is needed if a macro forces the expansion of subforms and moves pieces of the result into binding positions. Such a macro probably should not use `syntax-local-identifier-as-binding`, but it should first ensure that the macro use is in an expression context before forcing any subform expansions. Otherwise, the subform expansions could interact in unexpected ways with the use-site scopes of an enclosing definition context.

Use-site scopes associated with a first-class definition context are not stored directly in the compile-time environment frame for the definition context. Instead, they are stored in the closest frame that is not for a first-class definition context, so that the scopes are still tracked when the definition context is discarded (when the macro returns, typically). The scope for the definition context itself is similarly recorded in the closest such frame, so that `quote-syntax` can remove it, just like other binding scopes.

4.5 Modules and Phases

The `module` form creates a new scope for its body. More precisely, a `module` form creates an outside-edge scope and an inside-edge scope, like any other context that allows both definitions and macro expansion.

A `(module* name #f ....)` submodule form, where `#f` indicates that the enclosing module's bindings should be visible, creates an additional scope in the obvious way. For other `module*` and `module` submodule forms, the macro expander prevents access to the enclosing module's bindings by removing the two scopes of the enclosing module.

A module distinguishes bindings that have the same name but different phases. For example, `lambda` might have one meaning for run-time code within a module, but a different meaning for compile-time code within the same module. Furthermore, instantiating a module at a particular phase implies a phase shift in its syntax literals. Consider the module

```
(define x 1)
(define-for-syntax x 2)

(define id #'x)
(define-for-syntax id #'x)

(provide id (for-syntax id))
```

and suppose that the module is imported both normally and for compile time, the latter with a `s:` prefix. In a compile-time context within the importing module, both `id` and `s:id` will be bound to an identifier `x` that had the same scopes originally, but they should refer to different `x` bindings (in different module instances with different values).

Among the possibilities for distinguishing phases, having per-phase sets of scopes on an identifier makes the phase-shifting operation most natural. A local binding or macro expansion can add scopes at all phases, while `module` adds a distinct inside-edge scope to every phase (and the same outside-edge scope to all phases). Since every binding within a module is forced to have that module's phase-specific inside-edge scopes, bindings at different scopes will be appropriately distinguished.

Having a distinct "root" scope for each phase makes most local bindings phase-specific. That is, in

```
(define-for-syntax x 10)
(let ([x 1])
  (let-syntax ([y x])
    ....))
```

the `x` on the right-hand side of `let-syntax` sees the top-level phase-1 `x` binding, not the phase-0 local binding. This is a change from Racket's old approach to binding and phases, but the only programs that are affected are ones that would trigger an out-of-context error in the old system. Meanwhile, macros can construct identifiers that have no module scope, so out-of-context errors are still possible.

5. Implementation and Experience

Scope sets have an intuitive appeal as a model of binding, but a true test of the model is whether it can accommodate a Racket-scale use of macros—for constructing everything from simple syntactic abstractions to entirely new languages. Indeed, the set-of-scopes model was motivated in part by a fraying of Racket's old macro

expander at the frontiers of its implementation, e.g., for submodules (Flatt 2013).[2]

We released the new macro expander as part of Racket version 6.3 (released November 2015), while Racket developers started using the expander about four months earlier. Compared to the previous release, build times, memory use, and bytecode footprint were essentially unchanged compared to the old expander. Getting performance on par with the previous system required about two weeks of performance tuning, which we consider promising in comparison to a system that has been tuned over the past 15 years.

5.1 Initial Compatibility Results

At the time that Racket developers switched to the new expander, packages in Racket's main distribution had been adjusted to build without error (including all documentation), and most tests in the corresponding test suite passed; 43 out of 7501 modules failed. Correcting those failures before version 6.3 required small changes to accommodate the new macro expander.

Achieving the initial level of success required small changes to 15 out of about 200 packages in the distribution, plus several substantial macro rewrites in the core package:

- Changed macros in the core package include the `unit`, `class`, and `define-generics` macros, all of which manipulate scope in unusual ways.
- The Typed Racket implementation, which is generally sensitive to the details of macro expansion, required a handful of adjustments to deal with changed expansions of macros and the new scope-pruning behavior of `quote-syntax`.
- Most other package changes involve languages implementations that generate modules or submodules and rely on a non-composable treatment of module scopes by the old expander (which creates trouble for submodules in other contexts).

In about half of all cases, the adjustments for set-of-scopes expansion are compatible with the existing expander. In the other half, the macro adjustments were incompatible with the previous expander and the two separate implementations seem substantially easier to produce than one unified implementation.

Besides porting the main Racket distribution to a set-of-scopes expander, we tried building and testing all packages registered at `http://pkgs.racket-lang.org/`. There were 46 failures out of about 400 packages, as opposed to to 21 failures for the same set of packages with the previous Racket release. All of those packages were repaired before the version 6.3 release.

5.2 Debugging Support

Although the macro debugger (Culpepper and Felleisen 2010) has proven to be a crucial tool for macro implementors, binding resolution in Racket's old macro expander is completely opaque to macro implementers. When something goes wrong, the expander or macro debugger can report little more than "unbound identifier" or "out of context", because the process of replaying renamings and the encodings used for the renamings are difficult to unpack and relate to the programmer.

A set-of-scopes expander is more frequently in a position to report "unbound identifier, but here are the identifier's scopes, and here are some bindings that are connected to those scopes." In the case of ambiguous bindings, the expander can report the referencing identifier's scopes and the scopes of the competing bindings. These details are reported in a way similar to stack traces: subject to optimization and representation choices, and underspecified as a result, but invaluable for debugging purposes.

5.3 Scope Sets for JavaScript

Although the set-of-scopes model of binding was developed with Racket as a target, it is also intended as a more understandable model of macros to facilitate the creation of macro systems for other languages. In fact, the Racket implementation was not the first implementation of the model to become available. Based on an early draft of this report, Tim Disney revised the Sweet.js macro implementation for JavaScript (Disney et al. 2014; Disney et al. 2015)[3] to use scope sets even before the initial Racket prototype was complete. Disney reports that the implementation of hygiene for the macro expander is now "mostly understandable" and faster.

6. Model

We present a formal model of set-of-scope expansion following the style of Flatt et al. (2012). As a first step, we present a model where only run-time expressions are expanded, and implementations of macros are simply parsed. As a second step, we generalize the model to include phase-specific scope sets and macro expansion at all phases. The third step adds support for local expansion, and the fourth step adds first-class definition contexts. The model does not cover modules or top-level namespaces.

6.1 Single-Phase Expansion

Our macro-expansion model targets a language that includes with variables, function calls, functions, atomic constants, lists, and syntax objects:

$$
\begin{aligned}
\mathit{ast} &::= \mathit{var} \mid \textbf{App}(\mathit{ast}, \mathit{ast}, ...) \mid \mathit{val} \\
\mathit{var} &::= \textbf{Var}(\mathit{name}) \\
\mathit{val} &::= \textbf{Fun}(\mathit{var}, \mathit{ast}) \mid \mathit{atom} \mid \textbf{List}(\mathit{val}, ...) \mid \mathit{stx} \\
\mathit{stx} &::= \textbf{Stx}(\mathit{atom}, \mathit{ctx}) \mid \textbf{Stx}(\textbf{List}(\mathit{stx}, ...), \mathit{ctx}) \\
\mathit{id} &::= \textbf{Stx}(\mathit{sym}, \mathit{ctx}) \\
\mathit{atom} &::= \mathit{sym} \mid \mathit{prim} \mid .... \\
\mathit{sym} &::= \text{'}\mathit{name} \\
\mathit{name} &::= \text{a token such as } \texttt{x}, \texttt{egg}, \text{ or } \texttt{lambda}
\end{aligned}
$$

Since the model is concerned with macro expansion and programmatic manipulation of program terms, we carefully distinguish among

- *names*, which are abstract tokens;
- *variables*, which correspond to function arguments and references in an AST and are formed by wrapping a name as **Var**(*name*);
- *symbols*, which are values with respect to the evaluator and are formed by prefixing a name with a quote; and
- *identifiers*, which are also values, are formed by combining a symbol with a set of scopes, and are a subset of *syntax objects*.

For a further explanation of the distinctions among these different uses of names, see Flatt et al. (2012, section 3.2.1).

The model's evaluator is standard and relies on a δ function to implement primitives:

[2] For an example of a bug report about submodules, see problem report 14521 at `http://bugs.racket-lang.org/query/?debug=&database=default&cmd=view+audit-trail&cmd=view&pr=14521`. The example program fails with the old expander, due to problems meshing mark-oriented module scope with renaming-oriented local scope, but the example works with the set-of-scopes expander.

[3] See pull request 461 at `https://github.com/mozilla/sweet.js/pull/461`.

$\text{eval} : \mathit{ast} \to \mathit{val}$

$$\begin{aligned}
&\text{eval}[\![\textbf{App}(\mathit{ast}_{fun}, \mathit{ast}_{arg})]\!] &&= \text{eval}[\![\mathit{ast}_{body}[\mathit{var} \leftarrow \text{eval}[\![\mathit{ast}_{arg}]\!]]]\!] \\
&\quad \text{subject to } \text{eval}[\![\mathit{ast}_{fun}]\!] = \textbf{Fun}(\mathit{var}, \mathit{ast}_{body}) \\
&\text{eval}[\![\textbf{App}(\mathit{prim}, \mathit{ast}_{arg}, ...)]\!] &&= \delta(\mathit{prim}, \text{eval}[\![\mathit{ast}_{arg}]\!], ...) \\
&\text{eval}[\![\mathit{val}]\!] &&= \mathit{val}
\end{aligned}$$

Interesting primitives include the ones that manipulate syntax objects,

$$\mathit{prim} ::= \texttt{stx-e} \mid \texttt{mk-stx} \mid ....$$

where **`stx-e`** extracts the content of a syntax object, and **`mk-stx`** creates a new syntax object with a given content and the scopes of a given existing syntax object:

$$\begin{aligned}
&\delta(\texttt{stx-e}, \textbf{Stx}(\mathit{val}, \mathit{ctx})) &&= \mathit{val} \\
&\delta(\texttt{mk-stx}, \mathit{atom}, \textbf{Stx}(\mathit{val}, \mathit{ctx})) &&= \textbf{Stx}(\mathit{atom}, \mathit{ctx}) \\
&\delta(\texttt{mk-stx}, \textbf{List}(\mathit{stx}, ...), \textbf{Stx}(\mathit{val}, \mathit{ctx})) &&= \textbf{Stx}(\textbf{List}(\mathit{stx}, ...), \mathit{ctx})
\end{aligned}$$

Macro expansion takes a program that is represented as as a syntax object and produces a fully expanded syntax object. To evaluate the program, the syntax object must be parsed into an AST. The parser uses a resolve metafunction that takes an identifier and a binding store, Σ. The names `lambda`, `quote`, and `syntax`, represent the core syntactic forms, along with the implicit forms of function calls and variable reference:

$\text{parse} : \mathit{stx}\ \Sigma \to \mathit{ast}$

$$\begin{aligned}
&\text{parse}[\![\textbf{Stx}(\textbf{List}(\mathit{id}_{lam}, \mathit{id}, \mathit{stx}_{body}), \mathit{ctx}), \Sigma]\!] &&= \textbf{Fun}(\textbf{Var}(\text{resolve}[\![\mathit{id}, \Sigma]\!]), \\
&\qquad\qquad\qquad

A local macro binding via `let-syntax` is similar to an argument binding, but the compile-time environment records a macro transformer instead of a variable. The transformer is produced by using parse and then eval on the compile-time expression for the transformer. After the body is expanded, the macro binding is no longer needed, so the body expansion is the result.

$$\begin{aligned}
&\mathsf{expand}[\![\textbf{Stx}(\textbf{List}(id_{ls}, id, stx_{rhs}, stx_b), ctx), \xi, \Sigma]\!] = \mathsf{expand}[\![stx_{b2}, \xi_2, \Sigma_3]\!] \\
&\text{subject to } \mathsf{resolve}[\![id_{ls}, \Sigma]\!] = \texttt{let-syntax}, \\
&\qquad \mathsf{alloc\text{-}name}[\![\Sigma]\!] = \langle name_{new}, \Sigma_1\rangle, \mathsf{alloc\text{-}scope}[\![\Sigma_1]\!] = \langle scp_{new}, \Sigma_2\rangle, \\
&\qquad \mathsf{add}[\![id, scp_{new}]\!] = id_{new}, \Sigma_2{+}\{id_{new}{\to}name_{new}\} = \Sigma_3, \\
&\qquad \xi{+}\{name_{new}{\to}\mathsf{eval}[\![\mathsf{parse}[\![stx_{rhs}, \Sigma_3]\!]]\!]\} = \xi_2, \\
&\qquad \mathsf{add}[\![stx_b, scp_{new}]\!] = stx_{b2}
\end{aligned}$$

Finally, when the expander encounters an identifier that resolves to a binding mapped to a macro transformer, the transformer is applied to the macro use. Fresh scopes are generated to represent the use site, scp_u, and introduced syntax, scp_i, where the introduced-syntax scope is applied using flip to both the macro argument and result, where flip corresponds to an exclusive-or operation to leave the scope intact on syntax introduced by the macro (see below).

$$\begin{aligned}
&\mathsf{expand}[\![stx_{macapp}, \xi, \Sigma]\!] = \mathsf{expand}[\![\mathsf{flip}[\![stx_{exp}, scp_i]\!], \xi, \Sigma_2]\!] \\
&\text{subject to } stx_{macapp} = \textbf{Stx}(\textbf{List}(id_{mac}, stx_{arg}, ...), ctx), \\
&\qquad \xi(\mathsf{resolve}[\![id_{mac}, \Sigma]\!]) = val, \mathsf{alloc\text{-}scope}[\![\Sigma]\!] = \langle scp_u, \Sigma_1\rangle, \\
&\qquad \mathsf{alloc\text{-}scope}[\![\Sigma_1]\!] = \langle scp_i, \Sigma_2\rangle, \\
&\qquad \mathsf{eval}[\![\textbf{App}(val, \mathsf{flip}[\![\mathsf{add}[\![stx_{macapp}, scp_u]\!], scp_i]\!])]\!] = stx_{exp}
\end{aligned}$$

The only remaining case of expand is to recur for function-call forms, threading through the binding store using an accumulator-style expand* helper:

$$\begin{aligned}
&\mathsf{expand}[\![\textbf{Stx}(\textbf{List}(stx_{fun}, stx_{arg}, ...), ctx), \xi, \Sigma]\!] \\
&= \langle \textbf{Stx}(\textbf{List}(stx_{fun2}, stx_{arg2}, ...), ctx), \Sigma_1\rangle \\
&\text{subject to } \mathsf{expall}[\![(), (stx_{fun}\ stx_{arg}\ ...), \xi, \Sigma]\!] = \langle (stx_{fun2}\ stx_{arg2}\ ...), \Sigma_1\rangle
\end{aligned}$$

$$\underline{\mathsf{expall} : (stx\ ...)\ (stx\ ...)\ \xi\ \Sigma \to \langle (stx\ ...), \Sigma\rangle}$$

$$\begin{aligned}
&\mathsf{expall}[\![(stx_e\ ...), (), \xi, \Sigma]\!] = \langle (stx_e\ ...), \Sigma\rangle \\
&\mathsf{expall}[\![(stx_e\ ...), (stx_0\ stx_1\ ...), \xi, \Sigma]\!] = \mathsf{expall}[\![(stx_e\ ...\ stx_{e0}), (stx_1\ ...), \xi, \Sigma_1]\!] \\
&\text{subject to } \mathsf{expand}[\![stx_0, \xi, \Sigma]\!] = \langle stx_{e0}, \Sigma_1\rangle
\end{aligned}$$

For completeness, here are the add and flip metafunctions for propagating scopes to all parts of a syntax object, where $scp \oplus ctx$ adds scp to ctx if is not already in ctx or removes it otherwise:

$$\underline{\mathsf{add} : stx\ scp \to stx}$$

$$\begin{aligned}
&\mathsf{add}[\![\textbf{Stx}(atom, ctx), scp]\!] = \textbf{Stx}(atom, \{scp\} \cup ctx) \\
&\mathsf{add}[\![\textbf{Stx}(\textbf{List}(stx, ...), ctx), scp]\!] = \textbf{Stx}(\textbf{List}(\mathsf{add}[\![stx, scp]\!], ...), \{scp\} \cup ctx)
\end{aligned}$$

$$\underline{\mathsf{flip} : stx\ scp \to stx}$$

$$\begin{aligned}
&\mathsf{flip}[\![\textbf{Stx}(atom, ctx), scp]\!] = \textbf{Stx}(atom, scp \oplus ctx) \\
&\mathsf{flip}[\![\textbf{Stx}(\textbf{List}(stx, ...), ctx), scp]\!] = \textbf{Stx}(\textbf{List}(\mathsf{flip}[\![stx, scp]\!], ...), scp \oplus ctx)
\end{aligned}$$

To take a program from source to value, use expand, then parse, then eval.

6.2 Multi-Phase Expansion

To support phase-specific scope sets, we change the definition of ctx so that it is a mapping from phases to scope sets:

$$\begin{aligned}
&ph ::= integer \\
&ctx ::= \text{a mapping from } ph \text{ to } \overline{scp}
\end{aligned}$$

With this change, many metafunctions must be indexed by the current phase of expansion. For example, the result of resolve depends on the current phase:

$$\underline{\mathsf{resolve} : ph\ id\ \Sigma \to name}$$

$$\begin{aligned}
&\mathsf{resolve}_{ph}[\![\textbf{Stx}(\text{'}name, ctx), \Sigma]\!] = name_{biggest} \\
&\text{subject to } \Sigma(name) = \{\overline{scp}_{bind}{\leftarrow}name_{bind}, ...\}, \\
&\qquad \mathsf{biggest\text{-}subset}[\![ctx(ph), \{\overline{scp}_{bind}, ...\}]\!] = \overline{scp}_{biggest}, \\
&\qquad \{\overline{scp}_{bind}{\leftarrow}name_{bind}, ...\}(\overline{scp}_{biggest}) = name_{biggest} \\
&\mathsf{resolve}_{ph}[\![\textbf{Stx}(\text{'}name, ctx), \Sigma]\!] = name
\end{aligned}$$

Phase-specific expansion allows `let-syntax` to expand the compile-time expression for a macro implementation, instead of just parsing the expression. Note that the uses of expand and parse on the transformer expression are indexed by $ph{+}1$:

$$\underline{\mathsf{expand} : ph\ stx\ \xi\ \overline{scp}\ \Sigma \to \langle stx, \Sigma\rangle}$$

$$\begin{aligned}
&\mathsf{expand}_{ph}[\![\textbf{Stx}(\textbf{List}(id_{ls}, id, stx_{rhs}, stx_{body}), ctx), \xi, \overline{scp}_p, \Sigma]\!] \\
&= \mathsf{expand}_{ph}[\![stx_{body2}, \xi_2, \overline{scp}_{p2}, \Sigma_4]\!] \\
&\text{subject to } \mathsf{resolve}_{ph}[\![id_{ls}, \Sigma]\!] = \texttt{let-syntax}, \\
&\qquad \mathsf{alloc\text{-}name}[\![\Sigma]\!] = \langle name_{new}, \Sigma_1\rangle, \\
&\qquad \mathsf{alloc\text{-}scope}[\![\Sigma_1]\!] = \langle scp_{new}, \Sigma_2\rangle, \mathsf{add}_{ph}[\![id, scp_{new}]\!] = id_{new}, \\
&\qquad \Sigma_2{+}\{id_{new}{\to}name_{new}\} = \Sigma_3, \\
&\qquad \mathsf{expand}_{ph+1}[\![stx_{rhs}, \xi_{primitives}, \varnothing, \Sigma_3]\!] = \langle stx_{exp}, \Sigma_4\rangle, \\
&\qquad \xi{+}\{name_{new}{\to}\mathsf{eval}[\![\mathsf{parse}_{ph+1}[\![stx_{exp}, \Sigma_4]\!]]\!]\} = \xi_2, \\
&\qquad \mathsf{add}_{ph}[\![stx_{body}, scp_{new}]\!] = stx_{body2}, \{scp_{new}\} \cup \overline{scp}_p = \overline{scp}_{p2}
\end{aligned}$$

In addition to carrying a phase index, the revised expand takes a set of scopes created for bindings. Those scopes are the ones to be pruned from quoted syntax by the revised `syntax` expansion:

$$\begin{aligned}
&\mathsf{expand}_{ph}[\![\textbf{Stx}(\textbf{List}(id_{syntax}, stx), ctx), \xi, \overline{scp}_p, \Sigma]\!] \\
&= \langle \textbf{Stx}(\textbf{List}(id_{syntax}, stx_{pruned}), ctx), \Sigma\rangle \\
&\text{subject to } \mathsf{resolve}_{ph}[\![id_{syntax}, \Sigma]\!] = \texttt{syntax}, \\
&\qquad \mathsf{prune}_{ph}[\![stx, \overline{scp}_p]\!] = stx_{pruned}
\end{aligned}$$

The prune metafunction recurs through a syntax object to remove all of the given scopes at the indicated phase:

$$\underline{\mathsf{prune} : ph\ stx\ \overline{scp} \to stx}$$

$$\begin{aligned}
&\mathsf{prune}_{ph}[\![\textbf{Stx}(atom, ctx), \overline{scp}_p]\!] = \textbf{Stx}(atom, ctx{+}\{ph{\to}ctx(ph) \setminus \overline{scp}_p\}) \\
&\mathsf{prune}_{ph}[\![\textbf{Stx}(\textbf{List}(stx, ...), ctx), \overline{scp}_p]\!] \\
&= \textbf{Stx}(\textbf{List}(stx_{pruned}, ...), ctx{+}\{ph{\to}ctx(ph) \setminus \overline{scp}_p\}) \\
&\text{subject to } \mathsf{prune}_{ph}[\![stx, \overline{scp}_p]\!], ... = stx_{pruned}, ...
\end{aligned}$$

6.3 Local Expansion

Environment inspection via `syntax-local-value` and local expansion via `local-expand` are accommodated in the model essentially as in Flatt et al. (2012), but since local expansion can create bindings, the eval metafunction must consume and produce a binding store. The eval metafunction also must be index by the phase used for syntax operations.

Local expansion needs the current macro expansion's introduction scope, if any. In addition, local expansions that move identifiers into binding positions need `syntax-local-identifier-as-binding`, which requires information about scopes in the current expansion context. Local expansion, meanwhile, can create new such scopes. To support those interactions, eval and expand must both consume and produce scope sets for the current use-site scopes, and binding scopes must also be available for local expansion of `syntax` forms. To facilitate threading through all of that information, we define $\widehat{scp}$ as an optional current scope and $\widehat{\Sigma}$ as an extended store:

$\widehat{scp} ::= scp \mid \bullet$
$\widehat{\Sigma} ::= \langle \Sigma, \overline{scp}, \overline{scp} \rangle$

The second part of a $\widehat{\Sigma}$ tuple is a set of scopes to be pruned at `syntax` forms. The third part is a subset of those scopes that are the current expansion context's use-site scopes, which are pruned by `syntax-local-identifier-as-binding`. The different parts of a $\widehat{\Sigma}$ tuple vary in different ways: the Σ part is consistently threaded through evaluation and expansion, while the scope-set parts are stack-like for expansion and threaded through evaluation. In the case of a macro-application step, the scope-set parts of the tuple are threaded through expansion, too, more like evaluation.

In the model, the **`lvalue`**, **`lexpand`**, and **`lbinder`** primitives represent `syntax-local-value`, `local-expand`, and `syntax-local-identifier-as-binding`, respectively:

$\mathsf{eval} : ph\ ast\ \widehat{scp}\ \xi\ \widehat{\Sigma} \rightarrow \langle val, \widehat{\Sigma} \rangle$

$\mathsf{eval}_{ph}[\![\textbf{APP}(\textbf{`lvalue`}, ast_{id}), scp_i, \xi, \widehat{\Sigma}]\!] = \langle \xi(\mathsf{resolve}_{ph}[\![id_{result}, \Sigma_2]\!]), \widehat{\Sigma}_2 \rangle$
subject to $\mathsf{eval}_{ph}[\![ast_{id}, scp_i, \xi, \widehat{\Sigma}]\!] = \langle id_{result}, \widehat{\Sigma}_2 \rangle, \widehat{\Sigma}_2 = \langle \Sigma_2, _, _ \rangle$

$\mathsf{eval}_{ph}[\![\textbf{APP}(\textbf{`lexpand`}, ast, ast_{stops}), scp_i, \xi, \widehat{\Sigma}]\!] = \langle \mathsf{flip}_{ph}[\![stx_{exp}, scp_i]\!], \widehat{\Sigma}_4 \rangle$
subject to $\mathsf{eval}_{ph}[\![ast, scp_i, \xi, \widehat{\Sigma}]\!] = \langle stx, \widehat{\Sigma}_2 \rangle$,
$\mathsf{eval}_{ph}[\![ast_{stops}, scp_i, \xi, \widehat{\Sigma}_2]\!] = \langle \textbf{LIST}(id_{stop}, ...), \widehat{\Sigma}_3 \rangle$,
$\{var \rightarrow \mathsf{unstop}[\![\xi(var)]\!] \mid var \in \mathrm{dom}(\xi)\} = \xi_{unstops}$,
$\widehat{\Sigma}_3 = \langle \Sigma_3, _, _ \rangle, \mathsf{resolve}_{ph}[\![id_{stop}, \Sigma_3]\!], ... = name_{stop}, ...,$
$\xi_{unstops} + \{name_{stop} \rightarrow \textsc{Stop}(\xi_{unstops}(name_{stop}))\}\ ... = \xi_{stops}$,
$\mathsf{expand}_{ph}[\![\mathsf{flip}_{ph}[\![stx, scp_i]\!], \xi_{stops}, \widehat{\Sigma}_3]\!] = \langle stx_{exp}, \widehat{\Sigma}_4 \rangle$

$\mathsf{eval}_{ph}[\![\textbf{APP}(\textbf{`lbinder`}, ast_{id}), scp_i, \xi, \widehat{\Sigma}]\!] = \langle \mathsf{prune}_{ph}[\![id_{result}, \overline{scp}_{u2}]\!], \widehat{\Sigma}_2 \rangle$
subject to $\mathsf{eval}_{ph}[\![ast_{id}, scp_i, \xi, \widehat{\Sigma}]\!] = \langle id_{result}, \widehat{\Sigma}_2 \rangle, \widehat{\Sigma}_2 = \langle _, _, \overline{scp}_{u2} \rangle$

The implementation of **`lexpand`** uses a new $\textsc{Stop}(transform)$ transformer to make an identifier a stopping point for expansion while remembering the former *transform* mapping of the identifier. The unstop helper function strips away a STOP constructor:

$\mathsf{unstop} : transform \rightarrow transform$

$\mathsf{unstop}[\![\textsc{Stop}(transform)]\!] = transform$
$\mathsf{unstop}[\![transform]\!] = transform$

The expander must recognize STOP transformers to halt expansion at that point:

$\mathsf{expand} : ph\ stx\ \xi\ \widehat{\Sigma} \rightarrow \langle stx, \widehat{\Sigma} \rangle$

$\mathsf{expand}_{ph}[\![\textbf{STX}(\textbf{LIST}(id, stx, ...), ctx), \xi, \widehat{\Sigma}]\!] = \langle \textbf{STX}(\textbf{LIST}(id, stx, ...), ctx), \widehat{\Sigma} \rangle$
subject to $\widehat{\Sigma} = \langle \Sigma, _, _ \rangle, \xi(\mathsf{resolve}_{ph}[\![id, \Sigma]\!]) = \textsc{Stop}(_)$

The revised macro-application rule for expand shows how the use-site scopes component of $\widehat{\Sigma}$ is updated and how the current application's macro-introduction scope is passed to eval:

$\mathsf{expand}_{ph}[\![stx_{macapp}, \xi, \langle \Sigma, \overline{scp}_p, \overline{scp}_u \rangle]\!] = \langle stx_{result}, \widehat{\Sigma}_5 \rangle$
subject to $stx_{macapp} = \textbf{STX}(\textbf{LIST}(id_{mac}, stx_{arg}, ...), ctx)$,
$\xi(\mathsf{resolve}_{ph}[\![id_{mac}, \Sigma]\!]) = val, \mathsf{alloc\text{-}scope}[\![\Sigma]\!] = \langle scp_u, \Sigma_2 \rangle$,
$\mathsf{alloc\text{-}scope}[\![\Sigma_2]\!] = \langle scp_i, \Sigma_3 \rangle$,
$\langle \Sigma_3, \{scp_u\} \cup \overline{scp}_p, \{scp_u\} \cup \overline{scp}_u \rangle = \widehat{\Sigma}_3$,
$\mathsf{flip}_{ph}[\![\mathsf{add}_{ph}[\![stx_{macapp}, scp_u]\!], scp_i]\!] = stx_{macapp2}$,
$\mathsf{eval}_{ph}[\![\textbf{APP}(val, stx_{macapp2}), scp_i, \xi, \widehat{\Sigma}_3]\!] = \langle stx_{exp}, \widehat{\Sigma}_4 \rangle$,
$\mathsf{expand}_{ph}[\![\mathsf{flip}_{ph}[\![stx_{exp}, scp_i]\!], \xi, \widehat{\Sigma}_4]\!] = \langle stx_{result}, \widehat{\Sigma}_5 \rangle$

In contrast, the revised `lambda` rule shows how the pruning scope set is extended for expanding the body of the function, the use-site scope set is reset to empty, and all extensions are discarded in the expansion's resulting store tuple.

$\mathsf{expand}_{ph}[\![\textbf{STX}(\textbf{LIST}(id_{lam}, id_{arg}, stx_{bdy}), ctx), \xi, \langle \Sigma, \overline{scp}_p, \overline{scp}_u \rangle]\!]$
$= \langle \textbf{STX}(\textbf{LIST}(id_{lam}, id_{new}, stx_{bdy2}), ctx), \langle \Sigma_4, \overline{scp}_p, \overline{scp}_u \rangle \rangle$
subject to $\mathsf{resolve}_{ph}[\![id_{lam}, \Sigma]\!] = \texttt{lambda}$,
$\mathsf{alloc\text{-}name}[\![\Sigma]\!] = \langle name_{new}, \Sigma_1 \rangle$,
$\mathsf{alloc\text{-}scope}[\![\Sigma_1]\!] = \langle scp_{new}, \Sigma_2 \rangle, \mathsf{add}_{ph}[\![id_{arg}, scp_{new}]\!] = id_{new}$,
$\Sigma_2 + \{id_{new} \rightarrow name_{new}\} = \Sigma_3$,
$\xi + \{name_{new} \rightarrow \textsc{Var}(id_{new})\} = \xi_{new}$,
$\langle \Sigma_3, \{scp_{new}\} \cup \overline{scp}_p, \varnothing \rangle = \widehat{\Sigma}_3$,
$\mathsf{expand}_{ph}[\![\mathsf{add}_{ph}[\![stx_{bdy}, scp_{new}]\!], \xi_{new}, \widehat{\Sigma}_3]\!] = \langle stx_{bdy2}, \langle \Sigma_4, _, _ \rangle \rangle$

6.4 First-Class Definition Contexts

Supporting first-class definition contexts requires no further changes to the expand metafunction, but the eval metafunction must be extended to implement the **`new-defs`** and **`def-bind`** primitives, which model the `syntax-local-make-definition-context` and `syntax-local-bind-syntaxes` functions.

The **`new-defs`** primitive allocates a new scope to represent the definition context, and it also allocates a mutable reference to a compile-time environment that initially references the current environment. The two pieces are combined with a **DEFS** value constructor:

$\mathsf{eval}_{ph}[\![\textbf{APP}(\textbf{`new-defs`}), scp_i, \xi, \langle \Sigma, \overline{scp}_p, \overline{scp}_u \rangle]\!] = \langle \textbf{DEFS}(scp_{defs}, addr), \widehat{\Sigma}_3 \rangle$
subject to $\mathsf{alloc\text{-}scope}[\![\Sigma]\!] = \langle scp_{defs}, \Sigma_2 \rangle, \mathsf{alloc\text{-}def\text{-}env}[\![\Sigma_2]\!] = \langle addr, \Sigma_3 \rangle$,
$\langle \Sigma_2 + \{addr \rightarrow \xi\}, \{scp_{defs}\} \cup \overline{scp}_p, \overline{scp}_u \rangle = \widehat{\Sigma}_3$

The **`def-bind`** primitive works in two modes. In the first mode, it is given only a definition context and an identifier, and it creates a new binding for the identifier that includes the definition context's scope. The new binding is mapped to variable in an updated environment for definition context:

$\mathsf{eval}_{ph}[\![\textbf{APP}(\textbf{`def-bind`}, ast_{defs}, ast_{id}), scp_i, \xi, \widehat{\Sigma}]\!] = \langle 0, \langle \Sigma_6, \overline{scp}_{p3}, \overline{scp}_{u3} \rangle \rangle$
subject to $\mathsf{eval}_{ph}[\![ast_{defs}, scp_i, \xi, \widehat{\Sigma}]\!] = \langle \textbf{DEFS}(scp_{defs}, addr), \widehat{\Sigma}_2 \rangle$,
$\mathsf{eval}_{ph}[\![ast_{id}, scp_i, \xi, \widehat{\Sigma}_2]\!] = \langle id_{arg}, \widehat{\Sigma}_3 \rangle, \widehat{\Sigma}_3 = \langle \Sigma_3, \overline{scp}_{p3}, \overline{scp}_{u3} \rangle$,
$\mathsf{add}_{ph}[\![\mathsf{prune}_{ph}[\![\mathsf{flip}_{ph}[\![id_{arg}, scp_i]\!], \overline{scp}_{u3}]\!], scp_{defs}]\!] = id_{defs}$,
$\mathsf{alloc\text{-}name}[\![\Sigma_3]\!] = \langle name_{new}, \Sigma_4 \rangle, \Sigma_4 + \{id_{defs} \rightarrow name_{new}\} = \Sigma_5$,
$\Sigma_5(addr) = \xi_{defs}$,
$\Sigma_5 + \{addr \rightarrow \xi_{defs} + \{name_{new} \rightarrow \textsc{Var}(id_{defs})\}\} = \Sigma_6$

When **`def-bind`** is given an additional syntax object, it expands and evaluates the additional syntax object as a compile-time expression, and it adds a macro binding to the definition context's environment:

$\mathsf{eval}_{ph}[\![\textbf{APP}(\textbf{`def-bind`}, ast_{defs}, ast_{id}, ast_{stx}), scp_i, \xi, \widehat{\Sigma}]\!] = \langle 0, \widehat{\Sigma}_9 \rangle$
subject to $\mathsf{eval}_{ph}[\![ast_{defs}, scp_i, \xi, \widehat{\Sigma}]\!] = \langle \textbf{DEFS}(scp_{defs}, addr), \widehat{\Sigma}_2 \rangle$,
$\mathsf{eval}_{ph}[\![ast_{id}, scp_i, \xi, \widehat{\Sigma}_2]\!] = \langle id_{arg}, \widehat{\Sigma}_3 \rangle$,
$\mathsf{eval}_{ph}[\![ast_{stx}, scp_i, \xi, \widehat{\Sigma}_3]\!] = \langle stx_{arg}, \widehat{\Sigma}_4 \rangle$,
$\widehat{\Sigma}_4 = \langle \Sigma_4, \overline{scp}_{p4}, \overline{scp}_{u4} \rangle$,
$\mathsf{add}_{ph}[\![\mathsf{flip}_{ph}[\![stx_{arg}, scp_i]\!], scp_{defs}]\!] = stx_{arg2}$,
$\mathsf{expand}_{ph+1}[\![stx_{arg2}, \xi_{primitives}, \langle \Sigma_4, \varnothing, \varnothing \rangle]\!] = \langle stx_{exp}, \langle \Sigma_5, _, _ \rangle \rangle$,
$\mathsf{eval}_{ph}[\![\mathsf{parse}_{ph+1}[\![stx_{exp}, \Sigma_5]\!], \bullet, \xi, \langle \Sigma_5, \overline{scp}_{p4}, \varnothing \rangle]\!] = \langle val_{exp}, \widehat{\Sigma}_6 \rangle$,
$\widehat{\Sigma}_6 = \langle \Sigma_6, _, _ \rangle, \Sigma_6(addr) = \xi_{defs}$,
$\mathsf{add}_{ph}[\![\mathsf{prune}_{ph}[\![\mathsf{flip}_{ph}[\![id_{arg}, scp_i]\!], \overline{scp}_{u4}]\!], scp_{defs}]\!] = id_{defs}$,
$\mathsf{alloc\text{-}name}[\![\Sigma_6]\!] = \langle name_{new}, \Sigma_7 \rangle$,
$\Sigma_7 + \{id_{defs} \rightarrow name_{new}\} = \Sigma_8$,
$\langle \Sigma_8 + \{addr \rightarrow \xi_{defs} + \{name_{new} \rightarrow val_{exp}\}\}, \overline{scp}_{p4}, \overline{scp}_{u4} \rangle = \widehat{\Sigma}_9$

Note that **`def-bind`** in this mode defines a potentially recursive macro, since the definition context's scope is added to compile-time expression before expanding and parsing it.

Finally, a definition context is used to expand an expression by providing the definition context as an extra argument to **`lexpand`**. The implementation of the new case for **`lexpand`** is similar to the old one, but the definition context's scope is applied to the given syntax object before expanding it, and the definition context's environment is used for expansion.

$$
\begin{aligned}
&\mathsf{eval}_{ph}[\![\textbf{APP}(\texttt{\textbf{lexpand}}, ast_{expr}, ast_{stops}, ast_{defs}), scp_i, \xi, \widehat{\Sigma}]\!] = \langle stx_{exp2}, \widehat{\Sigma}_5\rangle \\
&\text{subject to } \mathsf{eval}_{ph}[\![ast_{expr}, scp_i, \xi, \widehat{\Sigma}]\!] = \langle stx, \widehat{\Sigma}_2\rangle, \\
&\qquad \mathsf{eval}_{ph}[\![ast_{stops}, scp_i, \xi, \widehat{\Sigma}_2]\!] = \langle \textbf{LIST}(id_{stop}, ...), \widehat{\Sigma}_3\rangle, \\
&\qquad \mathsf{eval}_{ph}[\![ast_{defs}, scp_i, \xi, \widehat{\Sigma}_3]\!] = \langle \textbf{DEFS}(scp_{defs}, addr), \widehat{\Sigma}_4\rangle, \\
&\qquad \widehat{\Sigma}_4 = \langle \Sigma_4, _, _\rangle, \Sigma_4(addr) = \xi_{defs}, \\
&\qquad \{var \rightarrow \mathsf{unstop}[\![\xi_{defs}(var)]\!] \mid var \in \mathsf{dom}(\xi_{defs})\} = \xi_{unstops}, \\
&\qquad \mathsf{resolve}_{ph}[\![id_{stop}, \Sigma_4]\!], ... = name_{stop}, ..., \\
&\qquad \xi_{unstops} + \{name_{stop} \rightarrow \textsc{Stop}(\xi_{unstops}(name_{stop}))\} \; ... = \xi_{stops}, \\
&\qquad \mathsf{expand}_{ph}[\![\mathsf{add}_{ph}[\![\mathsf{flip}_{ph}[\![stx, scp_i]\!], scp_{defs}]\!], \xi_{stops}, \widehat{\Sigma}_4]\!] = \langle stx_{exp}, \widehat{\Sigma}_5\rangle, \\
&\qquad \mathsf{flip}_{ph}[\![stx_{exp}, scp_i]\!] = stx_{exp2}
\end{aligned}
$$

7. Defining Hygiene

Most previous work on hygiene has focused on expansion algorithms, but some work addresses the question of what *hygiene* means independent of a particular algorithm. In his dissertation, Herman (2008) addresses the question through a type system that constrains and exposes the binding structure of macro expansions, so that α-renaming can be applied to unexpanded programs. More recently, Adams (2015) defines hygienic macro-expansion steps as obeying invariants that are expressed in terms of renaming via nominal logic (Pitts 2003), and the concept of equivariance plays an important role in characterizing hygienic macro transformers.

Since our notion of binding is based on scope sets instead of renaming, previous work on defining hygiene via renaming does not map directly into our setting. A related obstacle is that our model transforms a syntax object to a syntax object, instead of directly producing an AST; that difference is necessary to support local and partial expansion, which in turn is needed for definition contexts. A more technical obstacle is that we have specified expansion in terms of a meta-function (i.e., a big-step semantics) instead of as a rewriting system (i.e., a small-step semantics).

Adams's approach to defining hygiene nevertheless seems applicable to our notion of binding. We leave a full exploration for future work, but we can offer an informed guess about how that exploration will turn out.

Although our model of expansion does not incorporate renaming as a core concept, if we make assumptions similar to Adams (including omitting the `quote` form), then a renaming property seems useful and within reach. For a given set of scopes and a point during expansion (exclusive of macro invocations), the symbol can be swapped in every identifier that has a superset of the given set of scopes; such a swap matches the programmer's intuition that any variable can be consistently renamed within a binding region, which corresponds to a set of scopes. Hygienic expansion then means that the parse of the continued expansion after swapping is α-equivalent to what it would be without swapping. An individual transformer could be classified as hygienic based on all introduced identifiers having a fresh scope, so that they cannot bind any non-introduced identifiers; the fresh scope ensures an analog to Adams's equivariance with respect to binders.

Note that swapping `x` with `y` for the scope set $\{a_{def}, b_{intro1}\}$ would *not* produce an equivalent program for the expansion in section 3.5, because it would convert an ambiguous reference $\texttt{x}^{\{a_{def}, b_{intro1}, c_{intro2}\}}$ to an unambiguous $\texttt{y}^{\{a_{def}, b_{intro1}, c_{intro2}\}}$. This failure should not suggest that the pattern-matching macros in that example are non-hygienic in themselves, but that the (implicit) definition-context macro is potentially non-hygienic. That is, a macro in a definition context can introduce an identifier that is captured at the macro-use site, since the definition and use sites can be the same. That potential for non-hygienic expansion appears to be one of the trade-offs of providing a context that allows a mixture of mutually recursive macro and variable definitions.

If macro bindings are constrained to `letrec-syntax`, and if macro implementations are constrained use `syntax-case`, `free-identifier=?`, and `syntax->datum` (not `bound-identifier=?` or `datum->syntax`), then we expect that all expansion steps will be provably hygienic and all macro transformers will be provably hygienic by the definitions sketched above.

8. Other Related Work

While our work shares certain goals with techniques for representing resolved bindings, such as de Bruijn indices, higher-order abstract syntax (Pfenning and Elliott 1988), and nominal sets (Pitts 2013), those techniques seem to be missing a dimension that is needed to incrementally resolve bindings as introduced and manipulated by macros. Adams (2015) demonstrates how pairs of conventional identifiers provide enough of an extra dimension for hygienic macro expansion, but supporting `datum->syntax` would require the further extension of reifying operations on identifiers (in the sense of marks and renamings). Scope sets provides the additional needed dimension in a simpler way.

Scope graphs (Neron et al. 2015) abstract over program syntax to represent binding relationships—including support for constructs, such as modules and class bodies, that create static scopes different than nested lexical scopes. Binding resolution with macro expansion seems more dynamic, in that a program and its binding structure evolve during expansion; up-front scope graphs are not clearly applicable. Scope sets, meanwhile, do not explicitly represent import relationships, relying on macros that implement modular constructs to create scopes and bindings that reflect the import structure. Further work on scope graphs and scope sets seems needed to reveal the connections.

Stansifer and Wand (2014) build on the direction of Herman (2008) with Romeo, which supports program manipulations that respect scope by requiring that every transformer's type exposes its effect on binding. The resulting language is more general than Herman's macros, but transformers are more constrained than hygienic macros in Scheme and Racket.

9. Conclusion

Hygienic macro expansion is a successful, decades-old technology in Racket and the broader Scheme community. Hygienic macros have also found a place in some other languages, but the difficulties of specifying hygiene, understanding macro scope, and implementing a macro expander have surely been an obstacle to the broader adoption and use of hygienic macros. Those obstacles, in turn, suggest that our models of macro expansion have not yet hit the mark. Scope sets are an attempt to move the search for a model of macro expansion to a substantially different space, and initial results with Racket and JavaScript show that this new space is promising.

Acknowledgments

Thanks to Matthias Felleisen, Robby Findler, Sam Tobin-Hochstadt, Jay McCarthy, Ryan Culpepper, Michael Adams, Michael Ballantyne, Tim Disney, Simon Peyton Jones, Shriram Krishnamurthi, and Eelco Visser for discussions about set-of-scopes binding. Joshua Grams provided helpful editing suggestions. Anthony Carrico and Chris Brooks helpfully pointed out some typos. Thanks also to anonymous POPL reviewers for helping to improve the presentation. This work was supported by grants from the NSF.

References

Michael D. Adams. Towards the Essence of Hygiene. In *Proc. Principles of Programming Languages*, 2015.

Alan Bawden and Jonathan Rees. Syntactic Closures. In *Proc. Lisp and Functional Programming*, 1988.

William Clinger and Jonathan Rees. Macros that Work. In *Proc. Principles of Programming Languages*, 1991.

William D. Clinger. Hygienic Macros Through Explicit Renaming. *Lisp Pointers* 4(4), 1991.

Ryan Culpepper and Matthias Felleisen. Debugging Hygienic Macros. *Science of Computer Programming* 75(7), 2010.

Tim Disney et al. Sweet.js. 2015. `http://sweetjs.org/`

Tim Disney, Nathan Faubion, David Herman, and Cormac Flanagan. Sweeten your JavaScript: Hygienic Macros for ES5. In *Proc. Symposium on Dynamic Languages*, 2014.

R. Kent Dybvig, Robert Hieb, and Carl Bruggeman. Syntactic Abstraction in Scheme. *Lisp and Symbolic Computation* 5(4), 1993.

Matthew Flatt. Compilable and Composable Macros, You Want it *When?* In *Proc. International Conference on Functional Programming*, 2002.

Matthew Flatt. Submodules in Racket: You Want it *When*, Again? In *Proc. Generative Programming: Concepts and Experiences*, 2013.

Matthew Flatt, Ryan Culpepper, Robert Bruce Findler, and David Darais. Macros that Work Together: Compile-Time Bindings, Partial Expansion, and Definition Contexts. *Journal of Functional Programming* 22(2), 2012.

David Herman. Dissertation. PhD dissertation, Northeastern University, 2008.

Eugene Kohlbecker, Daniel P. Friedman, Matthias Felleisen, and Bruce Duba. Hygienic Macro Expansion. In *Proc. Lisp and Functional Programming*, 1986.

Eugene E. Kohlbecker and Mitchell Wand. Macro-by-Example: Deriving Syntactic Transformations from their Specifications. In *Proc. Principles of Programming Languages*, 1987.

Pierre Neron, Andrew P. Tolmach, Eelco Visser, and Guido Wachsmuth. A Theory of Name Resolution. In *Proc. European Symposium on Programming*, 2015.

Frank Pfenning and Conal Elliott. Higher-order Abstract Syntax. In *Proc. Programming Language Design and Implementation*, 1988.

Andrew M. Pitts. Nominal Logic, a First Order Theory of Names and Binding. *Information and Computation* 186(2), 2003.

Andrew M. Pitts. *Nominal Sets: Names and Symmetry in Computer Science*. Cambridge University Press, 2013.

Paul Stansifer and Mitch Wand. Romeo: a System for More Flexible Binding-Safe Programming. In *Proc. International Conference on Functional Programming*, 2014.